Signal Explorer scripting



SIGNAL EXPLORER

Scripting User's Guide

Basic Python for CAN / CAN FD and MCU testing

Application 0.1.0 | Document revision 1.1
November 2, 2026

Learn the editor, send and receive frames, validate responses, check counters and signals, and build repeatable MCU tests. Nine commented example files accompany this guide.

Remember
This guide uses the current injected **se** API. Python scripting is CAN/CAN FD only. The generic MCU protocol is fictional and must match your test firmware; the built-in virtual demo is a separate, smaller protocol.

Contents and example map

1 First run without hardware 3
2 Editor and execution controls 4
3 Python values, bytes and basic control flow 5
4 CAN API, timing and results 7
4.2 Explicit CAN-channel testing 9
5 Built-in simulation tests 10
6 MCU contract and bench templates 15
7 Limited UDS single-frame example 19
8 Reusable test patterns 21
9 Logging, replay and timestamps 22
9.1 TX lists and channel-aware evidence 23
10 Troubleshooting and practical limits 24
Appendix: source files, glossary and references 25

Choose the right example group

Example files	What must be available
01, 02, 03, 04 and 09	Built-in Virtual CAN with Inject demo traffic. Example 04 additionally requires CAN FD enabled before Connect.
05, 06, 07 and 08	A firmware/fixture matching section 6 (or a separately built mock peer). Default configuration guard prevents transmission.
can_demo_test.py	The existing continuous 321/221 demo opened by File Viewer > Open CAN Simulation Example.

Use the User Guide for adapter setup, wiring and pinouts. Use the Install Guide for Python/vendor driver deployment. This guide focuses on test code and evidence, not a replacement ECU specification.

1 First run without hardware

1. Launch Signal Explorer. In CAN / CAN FD select **Virtual (no hardware)**, check **Inject demo traffic** and Connect.

2. Choose **File Viewer > Open CAN Simulation Example**. The separate editor leaves the monitor available.

3. Set Line delay to zero and press Run. Look for TX 321 and RX 221 in the trace. IDs in this guide are hexadecimal.

4. The first payload is 00 01 02 03 04 05 06 07. The received payload must match; the next request increments each byte.

5. Try Pause, Resume and Stop. Stop ends the test without disconnecting. Normal simulated engine traffic continues.

6. For a finite exercise, open examples/scripts/mcu/01_engine_smoke.py. Run it and inspect the PASS checks and final PASSED result.

What the virtual peer actually does

ID / direction	Behavior
100 RX	Periodic DemoStatus. Byte zero increments modulo 256.
200 RX	Periodic EngineData: RPM, MAF, O2 voltage and lambda. Scales match examples/demo.dbc.
300 RX, FD mode	Periodic 32-byte FD telemetry with bitrate switching.
321 TX -> 221 RX	Eight-byte standard data-frame echo. FD requests receive FD replies with the same BRS flag.
Other MCU/UDS commands	Not implemented by this demo. The test templates need matching firmware or the private mock used by automated tests.

Remember
Virtual buses are in-process. Do not start a second Signal Explorer process as the virtual peer, and do not create a can.Bus inside a test script. Use the shared se connection.

2 Editor and execution controls

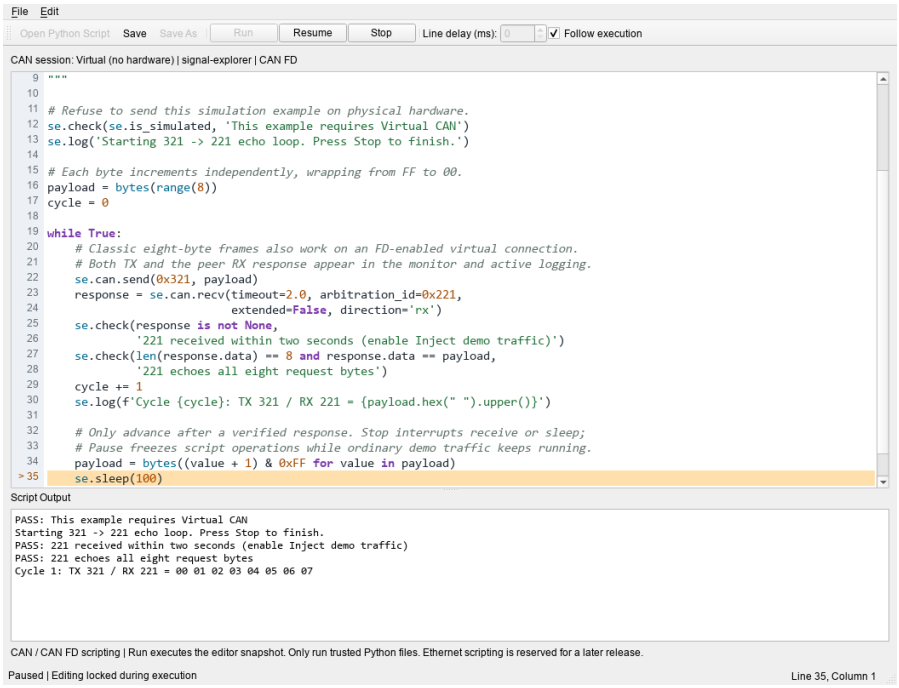


Figure 1: the editor paused during a simulated echo exchange.

Control	Use
New / Open / Save / Save As	Edit trusted .py files. Run executes the current editor snapshot, including unsaved changes. Save is atomic; source encoding is retained.
Run / Pause / Resume / Stop	One Python test at a time. Editing locks while it runs. Stop rejects further API bus requests; a blocked worker can be terminated after one second.
Line delay (ms)	0-1000 ms before each Python source line. Use a small value for learning; zero for timing checks. This changes script execution, unlike log-mode highlighting delay.
Follow execution	Scroll to the highlighted source line. Uncheck to inspect code elsewhere. Running is yellow, paused orange, failed red.
Script Output	se.log, print, PASS/FAIL checks and final state. Keeps up to 5000 blocks; not a durable report file.
Edit / Find	Undo/redo, cut/copy/paste, Select All, case-sensitive Find/Replace; Tab/Shift+Tab indent, Enter preserves indentation.

Common shortcuts: Ctrl+O open, Ctrl+S save, Ctrl+Shift+S Save As, Ctrl+Z undo, Ctrl+Y redo and Ctrl+F Find. Closing an active editor stops its script before releasing the window.

A fresh worker process is not a security sandbox: Python retains the user's file/network privileges. Run trusted scripts. Direct driver calls, unmanaged threads or child programs bypass the intended shared CAN controls.

3 Python values and CAN bytes

Python runs statements from top to bottom. A variable names a value. A comment starts with #. Hexadecimal notation starts with 0x; 0x321 means the same ID shown as 321 in the GUI. Decimal 321 is a different ID.

```
# Integers, strings and Boolean flags.
request_id = 0x321
name = "Echo test"
extended = False

# A byte is an integer from 0 to 255 (00 to FF).
payload = bytes([0x12, 0x34, 0, 1, 2, 3, 4, 5])
first_byte = payload[0]       # Indexing starts at zero: 0x12.
first_two = payload[0:2]     # Slice includes 0, excludes 2.
count = len(payload)         # Eight bytes.
se.log(payload.hex(" "))     # 12 34 00 01 02 03 04 05

# Byte order must match the MCU's documented payload.
big = int.from_bytes(first_two, "big")       # 0x1234 = 4660.
little = int.from_bytes(first_two, "little") # 0x3412 = 13330.
encoded = (4660).to_bytes(2, "big")          # b"\x12\x34"

# Wrap each byte independently instead of exceeding FF.
incremented = bytes((value + 1) & 0xFF for value in payload)
se.check(count == 8, "Eight-byte payload")
```

Choose a representation

bytes is immutable and convenient for frames. **bytearray** is mutable for building a payload, then bytes(buffer) freezes it. Lists of integers also work with se.can.send. A text string such as "DE AD" is not a byte array: use bytes.fromhex("DE AD").

Remember
The script API frame data is bytes. Verify a received frame is not None and its length is correct before indexing fields. A missing frame is a timeout, not an all-zero payload.

3.1 Decisions, loops and functions

Indentation defines a block. Use four spaces. A colon starts an if, for, while or function block. == compares; = assigns. Functions let several tests reuse one validated operation.

```
def decode_rpm(data):
    # A helper returns a result to its caller.
    return int.from_bytes(data[0:2], "little") * 0.25

for sample in range(3):
    frame = se.can.recv(2.0, arbitration_id=0x200)
    if frame is None:
        raise RuntimeError("No engine frame")
    se.check(len(frame.data) == 8, "Engine frame length")
    rpm = decode_rpm(frame.data)
    se.log(f"Sample {sample + 1}: {rpm:.0f} rpm")
    se.sleep(100)

# An intentional loop uses API waits so Pause and Stop stay responsive.
# while True:
#     frame = se.can.recv(timeout=None, arbitration_id=0x200)
#     se.log(frame.data.hex(" "))
```

Errors and test outcomes

An unhandled exception ends the run as FAILED. se.check(False, description) prints FAIL and raises AssertionError. Catch an exception only when the protocol expects it and you have a deliberate recovery plan. Catching a failed se.check does not erase that failure from the final result.

PASSED means at least one check ran and no recorded check failed. COMPLETED means a run ended with no checks. STOPPED means it was interrupted; it does not certify the remaining steps. A while True test has no normal end and usually finishes STOPPED after the operator presses Stop.

Imports and helpers

Standard modules such as struct, json and pathlib are available. Saved scripts can import nearby helper files; each run starts a fresh process/namespace. The se object is injected, so do not import se or install a package with that name. input() is unsupported and raises EOFError.

4 Signal Explorer CAN scripting API

API	Meaning and units
se.protocol	Currently "CAN". Selecting another main tab does not switch the Python transport.
se.session	Run-start metadata: interface, adapter, native_channel, channels mapping, fd, bitrates and simulated flag. It does not reconfigure hardware.
se.is_simulated	True only for the connected Virtual CAN session. Use as a guard for built-in demo scripts.
se.log(value)	Append a message to Script Output. print(...) is captured too.
se.check(condition, description)	Print PASS/FAIL; a false condition normally terminates execution with an assertion.
se.sleep(milliseconds)	Interruptible, pause-aware wait. Not a hardware scheduler.
se.can.send(id, data, ...)	Send once. Options: extended=False, fd=False, brs=False, channel=None (enabled label or native ID). Uses the connected adapter, channel and FD validation. Raises on failure.
se.can.recv(timeout=1.0, ...)	Wait in seconds; None waits until a match or Stop. Returns CanFrame or None. Filters: arbitration_id, extended, direction (rx, tx or all), channel (enabled label/native ID). Default direction is rx.

```
se.can.send(0x321, bytes(range(8)))
reply = se.can.recv(2.0, arbitration_id=0x221,
                    extended=False, direction="rx")
se.check(reply is not None, "Response received")
se.check(reply.data == bytes(range(8)), "Correct echo")
```

CanFrame fields: arbitration_id, data, timestamp, channel, direction, is_extended_id, is_fd, bitrate_switch, is_error_frame and is_remote_frame. The receive API excludes error/remote events. There is no built-in DBC signal lookup, periodic-send task, adapter-switch command or full UDS client in se.

4.1 Matching, timing and test results

A local TX entry is not an MCU response

A send uses the main application path, which records one local accepted TX. Receiving with direction="rx" prevents that local echo from satisfying a response check. Also match the response ID, standard/extended format, length, opcode, sequence or echoed payload as appropriate.

Do not accept stale data

A response can already be queued when recv starts. The script has an independent 2000-event mailbox; unmatched events are consumed as it searches. Use a transaction field to distinguish a new reply from an old one. Do not overlap requests to different response IDs unless you implement coordinated dispatch: one filtered receive can consume another response.

Use bounded retries

Send a command once, then wait for its response under the firmware's deadline. A retry is a new action: repeated reset, erase or output commands may have side effects. The GPIO template waits up to ten 0.2-second receives for a matching transaction, rather than blindly resending the command.

Pause and line delay

se.sleep and receive timeouts exclude time spent paused. They do not subtract ordinary processing or line delay. Python time.monotonic/time.sleep use real elapsed time and do not inherit this pause model. For a period test, use message timestamps and zero line delay; do not pause in the measurement interval unless that is part of the test definition.

Timing claims

Loops that send then sleep add processing time to the nominal sleep. A se.sleep(100) request is not a guarantee of an exact 100 ms hardware period. Use the main repeated-send control or an appropriate hardware scheduler when exact traffic timing is the test stimulus. Log scheduling delay and verify wire timing independently.

Remember
Trace pause/filter does not pause scripts or logging. Script Pause does not stop the MCU, other manual transmit controls, replay or periodic demo traffic. Stop cannot retract an already accepted frame or guarantee MCU cleanup.

4.2 Test explicitly enabled CAN channels

Enable the needed channels in the main CAN window before Connect. The script shares that connection; it does not open adapters or change bitrates. Optional channel accepts a label such as CAN1 or a native channel identifier. Omitting send channel uses the current main TX destination; omitting receive channel accepts any enabled source.

```
# Inspect the run-start snapshot before writing a channel-specific test.
for port in se.session.get("channels", []):
    se.log(port)

# For a correctly wired CAN1-to-CAN2 bench loop only.
# Configure both ports at 500 kbit/s Classic CAN in the main window.
labels = {p["label"] for p in se.session.get("channels", [])}
se.check({"CAN1", "CAN2"} <= labels, "Both ports enabled")
payload = bytes.fromhex("A5 5A 01 02 03 04 05 06")
se.can.send(0x5A1, payload, channel="CAN1")
reply = se.can.recv(1.0, arbitration_id=0x5A1,
                    extended=False, direction="rx", channel="CAN2")
se.check(reply is not None, "CAN2 received frame")
se.check(reply.data == payload, "Loopback bytes match")
```

Native identifiers and labels

se.session["channels"] contains each port native_channel, label and bus_channel mapping. CanFrame.channel remains the native identifier as a string; it is not always the display label CAN1. Match through recv(channel=...) or consult the snapshot instead of assuming a numeric offset. Disabled destinations are rejected; sends are not broadcast.

FD and virtual test boundaries

Existing scripts without channel arguments continue to work. FD and BRS still require an FD-enabled connection with compatible timing on every enabled port. The built-in single Virtual CAN demo is not an electrical two-port loopback. Use its 321/221 echo examples for hardware-free checks; use the example above only with the corresponding bench wiring.

One VN1630A CAN1-to-CAN2 Classic CAN frame was physically verified November 2. FD routing and multi-channel mailbox/recording behavior use virtual/mock tests; no physical FD result is implied.

5 Continuous 321/221 echo demo

The menu example starts at 00 01 02 03 04 05 06 07 and increments every byte only after a verified response. IDs are standard hexadecimal 321 and 221. The built-in peer runs only on Virtual CAN with Inject demo traffic enabled.

```
se.check(se.is_simulated, "Use Virtual CAN with demo traffic")
payload = bytes(range(8))
cycle = 0
while True:
    se.can.send(0x321, payload)
    reply = se.can.recv(2.0, arbitration_id=0x221,
                        extended=False, direction="rx")
    se.check(reply is not None, "221 arrived")
    se.check(len(reply.data) == 8, "Eight-byte response")
    se.check(reply.data == payload, "Exact echo")
    cycle += 1
    se.log(f"Cycle {cycle}: {payload.hex(' ')}")
    payload = bytes((value + 1) & 0xFF for value in payload)
    se.sleep(100)
```

Expected trace

Cycle	TX 321 and RX 221 data
1	00 01 02 03 04 05 06 07
2	01 02 03 04 05 06 07 08
3	02 03 04 05 06 07 08 09
After FF	Each byte wraps independently to 00; this is not one 64-bit counter.

The normal periodic 100/200 traffic remains present, plus 300 in FD mode. The standalone source copy is examples/scripts/can_demo_test.py. This manual's shorter listing shows the same control pattern; the packaged menu example also includes extra comments and logging.

5.1 Telemetry smoke test

Run with the built-in virtual demo. This checks frame arrival, length and decoded ranges for five samples. It is a useful first functional check before testing an MCU response protocol.

Source: examples/scripts/mcu/01_engine_smoke.py

```
"""Built-in Virtual CAN: decode five EngineData frames."""
se.check(se.is_simulated, 'Use Virtual CAN with demo traffic enabled')
for sample in range(5):
    frame = se.can.recv(2.0, arbitration_id=0x200, extended=False)
    se.check(frame is not None, 'EngineData arrived')
    se.check(len(frame.data) == 8, 'Expected eight bytes')
    # Little-endian unsigned 16-bit values, scaled as in demo.dbc.
    rpm = int.from_bytes(frame.data[0:2], 'little') * 0.25
    maf = int.from_bytes(frame.data[2:4], 'little') * 0.01
    o2 = int.from_bytes(frame.data[4:6], 'little') * 0.001
    se.check(800 <= rpm <= 2800, 'RPM within demo limits')
    se.check(10 <= maf <= 50, 'MAF within demo limits')
    se.check(0.10 <= o2 <= 0.80, 'O2 within demo limits')
    se.log(f'Sample {sample + 1}: {rpm:.0f} rpm, {maf:.2f} g/s')
```

What the bytes mean

Bytes	Demo scaling
0-1	RPM: unsigned little-endian integer multiplied by 0.25.
2-3	MAF: unsigned little-endian integer multiplied by 0.01 g/s.
4-5	O2 voltage: unsigned little-endian integer multiplied by 0.001 V.
6-7	Lambda: unsigned little-endian integer multiplied by 0.001 (not used in this short smoke test).

These are the synthetic examples/demo.dbc values, not standardized OBD-II PIDs. Loading that DBC changes the monitor display; the script decodes raw bytes itself. For real firmware, copy the correct endian/sign/scale/offset and acceptance limits from the actual interface specification.

5.2 Heartbeat and counter continuity

A heartbeat test can establish that firmware remains alive and advances its sequence field. This listing uses the demo status byte and a broad period tolerance; adapt both fields and limits for a physical MCU.

Source: examples/scripts/mcu/02_heartbeat_counter.py

```
"""Built-in Virtual CAN: monitor counter continuity and period."""
se.check(se.is_simulated, 'Use Virtual CAN with demo traffic enabled')
previous = None
for sample in range(8):
    frame = se.can.recv(2.0, arbitration_id=0x100, extended=False)
    se.check(frame is not None, 'Heartbeat received')
    se.check(len(frame.data) == 8, 'Heartbeat length')
    if previous is not None:
        # Modulo arithmetic accepts the FF -> 00 rollover.
        delta = (frame.data[0] - previous.data[0]) & 0xFF
        se.check(delta == 1, 'No missing or repeated demo counter')
        period = frame.timestamp - previous.timestamp
        # Broad simulation tolerance, not a production timing limit.
        se.check(0.02 <= period <= 0.50, 'Demo heartbeat period')
    previous = frame
se.log('Eight consecutive heartbeat frames checked')
```

Interpreting failures

A counter delta of zero may be a duplicate; a delta greater than one can indicate a missed application frame, transport loss or a different publisher. It does not identify which layer dropped data. Check the trace, logger, driver diagnostics and an independent peer before attributing the failure to firmware.

The period uses recorded RX timestamps, rather than how fast the editor received callbacks. This matters when the UI is busy or buffered. The demo uses one counter publisher; another sender using ID 100 on the same virtual channel would invalidate that assumption.

Remember
For a production test define startup allowance, expected period and jitter, counter width, wrap rule, reset behavior, timeout and whether duplicate frames are permitted. The demonstration limits are not an automotive acceptance standard.

5.3 Payload boundary and walking-bit tests

All-zero, all-one and alternating patterns exercise payload handling. Walking a single set bit through each byte can expose truncation, byte swapping and incorrect masks. The virtual peer echoes the entire frame.

Source: examples/scripts/mcu/03_echo_vectors.py

```
"""Built-in Virtual CAN: boundary and walking-bit payload tests."""
se.check(se.is_simulated, 'Use Virtual CAN with demo traffic enabled')
vectors = [bytes(8), bytes([0xFF] * 8), bytes([0xAA, 0x55] * 4)]
vectors += [bytes([1 << bit] * 8) for bit in range(8)]
for index, payload in enumerate(vectors):
    se.can.send(0x321, payload)
    reply = se.can.recv(2.0, arbitration_id=0x221, extended=False)
    se.check(reply is not None, 'Echo response received')
    se.check(reply.data == payload, f'Eight-byte vector {index + 1}')
    se.log(f'321 -> 221: {payload.hex(" ").upper()}')
    se.sleep(100)  # Milliseconds; Pause/Stop interrupt this wait.
```

Extend the vector list deliberately

Add command-specific boundary values such as minimum/maximum sensor counts, signed limits and reserved bits only when the firmware contract permits them. An echo checks communication/data handling; it does not prove that an actuator moved, an analog input was sampled or a diagnostic routine completed.

For a signed 16-bit field use int.from_bytes(data, "little", signed=True) and check the expected sign. For a bit field use (value & mask) != 0. Compare complete responses when possible and validate reserved fields separately if the specification defines their values.

Remember
The built-in 321 responder accepts exactly eight data bytes and standard IDs. It does not echo arbitrary IDs, extended frames or every CAN FD payload length.

5.4 CAN FD and controlled timeouts

Enable CAN FD before connecting Virtual CAN. This checks 32-byte telemetry, then an eight-byte FD/BRS request-response exchange. The echo peer's eight-byte rule still applies.

Source: examples/scripts/mcu/04_can_fd_echo.py

```
"""Built-in Virtual CAN FD: 32-byte telemetry and eight-byte FD echo."""
se.check(se.is_simulated, 'Use Virtual CAN with demo traffic enabled')
se.check(se.session.get('fd'), 'Enable CAN FD before connecting')
telemetry = se.can.recv(2.0, arbitration_id=0x300, extended=False)
se.check(telemetry is not None, 'FD telemetry arrived')
se.check(telemetry.is_fd and len(telemetry.data) == 32, '32-byte FD frame')

# The built-in echo peer accepts exactly eight bytes, including in FD mode.
payload = bytes(range(8))
se.can.send(0x321, payload, fd=True, brs=True)
reply = se.can.recv(2.0, arbitration_id=0x221, extended=False)
se.check(reply is not None, 'FD echo response arrived')
se.check(reply.is_fd and reply.bitrate_switch, 'FD/BRS flags retained')
se.check(reply.data == payload, 'Eight-byte FD payload echoed')
```

A separate no-response exercise

Source: examples/scripts/mcu/09_receive_timeout.py

```
"""Built-in Virtual CAN: demonstrate a controlled receive timeout."""
se.check(se.is_simulated, 'Use a private Virtual CAN demo session')
# The built-in demo does not emit 7FF. This sends no frames.
frame = se.can.recv(0.2, arbitration_id=0x7FF, extended=False)
se.check(frame is None, 'Expected no 7FF response in the demo')
se.log('Timeout handling passed; this does not diagnose an ECU fault')
```

The timeout exercise sends nothing. None means no matching event within the receive deadline; it is not proof that a real MCU rejected a command. FD payload lengths supported by the application are 0-8, 12, 16, 20, 24, 32, 48 and 64. This build validates mode/length before sending; hardware must support the selected rates and BRS.

6 Define your MCU test contract

The next templates use a **fictional MCU application protocol**. They are examples for firmware you control, not Astemo production requirements or universal automotive CAN IDs. The built-in demo does not implement it.

Standard ID / bytes	Fictional firmware contract
500 request, 8 bytes	Byte 0 opcode; byte 1 argument; bytes 2-6 reserved zero; byte 7 transaction (0-255).
Opcode 01: set GPIO	Argument is 0 or 1. Reply 501: byte 0=81, byte 1=acknowledged state, bytes 2-3=unsigned little-endian ADC, bytes 4-6 zero, byte 7=transaction.
Opcode 03: query boot count	Reply 502: byte 0=B0, bytes 1-2=unsigned little-endian retained boot counter, bytes 3-6 zero, byte 7=transaction.
Opcode 02: reset	Firmware retains/increments the boot counter and returns the same 502 boot report after restart, echoing the reset transaction.
Unsupported opcode 7E	Reply 501: FF 11 00 00 00 00 00 transaction. This is a custom application response, not UDS.
ADC fixture assumption	Independent known midpoint analog stimulus; 12-bit counts, nominal 3.3 V reference, example acceptance 1000-3000. GPIO acknowledgement is not physical pin readback.
700 request / 708 response	Separate optional UDS teaching endpoint: normal addressing, Classic CAN single frames, DID 1234 with exactly two data bytes. Not the 500 protocol.

Before enabling a template

1. Confirm your interface specification, rates, IDs, byte order, reset cause/retention behavior and transaction handling. Change the template to match, rather than guessing.

2. Implement/verify the peer firmware or a private mock. Check a simple request at the independent peer before a sequence test.

3. Define fixture stimulus and expected response limits. Arrange manual/firmware-timeout cleanup for outputs, since stopping a Python process does not restore the MCU.

4. Set FIRMWARE_CONTRACT_CONFIRMED = True only in your reviewed copy, then Save As under a project-specific name. Leave the supplied templates unchanged for reference.

6.1 GPIO acknowledgement and ADC limits

Requires the section 6 firmware/fixture. This sequence commands 0, 1, then 0, checks the transaction/acknowledgement and validates an independent midpoint ADC stimulus.

Source: examples/scripts/mcu/05_gpio_adc_template.py

```
"""MCU TEMPLATE: requires the guide's fictional command/response firmware."""
FIRMWARE_CONTRACT_CONFIRMED = False
se.check(FIRMWARE_CONTRACT_CONFIRMED, 'Review the MCU contract before TX')

def set_output(state, transaction):
    # 500: opcode, output state, five reserved bytes, transaction.
    request = bytes([0x01, state, 0, 0, 0, 0, 0, transaction])
    se.can.send(0x500, request)
    for attempt in range(10):
        reply = se.can.recv(0.2, arbitration_id=0x501, extended=False)
        if reply is None:
            continue
        se.check(len(reply.data) == 8, 'MCU response has eight bytes')
        if reply.data[7] == transaction and reply.data[0] == 0x81:
            return reply
    raise RuntimeError('No matching GPIO acknowledgement in ten waits')

for transaction, state in enumerate((0, 1, 0), start=1):
    reply = set_output(state, transaction)
    se.check(reply.data[1] == state, 'Commanded output state acknowledged')
    adc = int.from_bytes(reply.data[2:4], 'little')
    se.check(0 <= adc <= 4095, 'ADC fits a 12-bit reading')
    # Assumes a known midpoint stimulus on the independent ADC input.
    se.check(1000 <= adc <= 3000, 'ADC within this fixture acceptance band')
    voltage = adc * 3.3 / 4095
    se.log(f'Output={state}, ADC={adc}, nominal voltage={voltage:.3f} V')
    se.sleep(100)
# Stop does not reset MCU outputs. Use firmware timeout/manual cleanup too.
```

The voltage calculation assumes the nominal reference and scaling shown; it is not a calibrated voltmeter. To prove actual GPIO output, add independent fixture readback. The ten receive waits bound transaction matching to approximately two seconds plus processing; individual waits exclude Pause.

6.2 MCU reset and boot recovery

Requires firmware that retains a boot count and echoes the reset transaction in its post-boot report. This is an application reset command, not the UDS ECUReset service.

Source: examples/scripts/mcu/06_reset_recovery_template.py

```
"""MCU TEMPLATE: query boot count, request reset, confirm new boot report."""
FIRMWARE_CONTRACT_CONFIRMED = False
se.check(FIRMWARE_CONTRACT_CONFIRMED, 'Review the MCU contract before TX')

def boot_report(transaction, timeout):
    for attempt in range(3):
        frame = se.can.recv(timeout, arbitration_id=0x502, extended=False)
        se.check(frame is not None, 'Boot report received')
        se.check(len(frame.data) == 8, 'Boot report length')
        if frame.data[0] == 0xB0 and frame.data[7] == transaction:
            return int.from_bytes(frame.data[1:3], 'little')
    raise RuntimeError('Boot report transaction did not match')

# Fictional command 03 queries the current retained boot counter.
se.can.send(0x500, [0x03, 0, 0, 0, 0, 0, 0, 1])
before = boot_report(1, 1.0)
# Fictional command 02 resets; firmware echoes transaction after restart.
se.can.send(0x500, [0x02, 0, 0, 0, 0, 0, 0, 2])
after = boot_report(2, 3.0)
se.check(after == ((before + 1) & 0xFFFF), 'One MCU reboot observed')
se.log(f'Boot count changed from {before} to {after}')
```

Define the reset acceptance criteria

Specify what persists across reset and what the board reports after startup. A counter alone should not be your only production proof: add reset-reason bits, expected firmware version, heartbeat restart and output-default checks if available.

Each boot_report call allows up to three receives to discard stale transactions. The example uses a three-second per-receive timeout after reset, so it is not a strict single three-second overall reboot budget. For a fixed end-to-end deadline, use a single receive when the firmware guarantees transaction ordering, or a pause-aware deadline helper designed for your runner.

Remember
The current scripting API does not control ECU power, external reset pins, a programmer or a USB relay. Those require a separate approved fixture integration. Reset tests should not be treated as a flashing/programming feature.

6.3 Expected rejection and negative tests

A negative test expects a documented rejection while the MCU remains operational. This example sends the reserved unsupported opcode from section 6 and checks the complete rejection prefix and transaction.

Source: examples/scripts/mcu/08_negative_command_template.py

```
"""MCU TEMPLATE: verify a documented unsupported-opcode response."""
FIRMWARE_CONTRACT_CONFIRMED = False
se.check(FIRMWARE_CONTRACT_CONFIRMED, 'Review the MCU contract before TX')
transaction = 9
# 7E is reserved as unsupported in this fictional firmware protocol.
se.can.send(0x500, [0x7E, 0, 0, 0, 0, 0, 0, transaction])
for attempt in range(5):
    reply = se.can.recv(0.2, arbitration_id=0x501, extended=False)
    if reply is None:
        continue
    se.check(len(reply.data) == 8, 'Negative response length')
    if reply.data[7] == transaction:
        se.check(reply.data[:2] == bytes([0xFF, 0x11]), 'Unsupported opcode')
        break
else:
    raise RuntimeError('No matching rejection response')
# FF/11 here is our fictional application response, not a UDS packet.
```

Useful next negative-test cases

After defining the firmware behavior, test an out-of-range application value, unsupported mode, wrong transaction or missing prerequisite. Confirm both the rejection response and unchanged state. Do not equate application-level rejection with malformed CAN frames or electrical error injection; se sends validated frames and has no physical fault-injection API.

Keep a distinct test case for no response: a timeout can mean an intentionally silent command, a bus problem, an ID mismatch or a firmware defect. A test passes only when its actual expected behavior is explicit. Retrying an invalid command without a bound can hide the first failure.

7 UDS over CAN: scope and framing

UDS describes diagnostic services; ISO-TP carries their payloads over CAN. A small payload can use one Single Frame. Longer transfers need segmentation and flow control. Signal Explorer currently exposes raw CAN frames, not a complete UDS/ISO-TP stack. [R1,R2]

Teaching example: read one two-byte DID

Layer / bytes	Interpretation
Request ID 700	Example physical 11-bit address; not a required industry-wide ECU ID.
03 22 12 34 00 00 00 00	Classic CAN SF length 3, then UDS ReadDataByIdentifier (22), then example DID 1234. Remaining bytes are padding.
Response ID 708	Example response address to configure on the matching firmware.
05 62 12 34 12 34 00 00	SF length 5; positive SID 62, echoed DID 1234, then this fictional DID's two-byte big-endian value 1234.
03 7F 22 78 00 00 00 00	Negative-response payload 7F 22 78 means response pending. Wait for the final reply instead of resending the request. [R3]

The positive ReadDataByIdentifier response contains the requested DID and its data. The data encoding/length comes from the ECU specification, not from service 22 alone. [R1]

What this example intentionally does not implement

No First/Consecutive/Flow-Control frames, extended/mixed addressing, CAN FD SF escape format, automatic diagnostic session management, TesterPresent scheduling, P2/P2* negotiation, SecurityAccess, DTC parsing, flashing or transfer routines. A multi-frame response fails explicitly rather than being partially accepted. A VIN read usually needs more than this two-byte teaching contract.

For an expanded UDS client, keep transport integration on the shared application connection. Opening a second python-can adapter handle inside the script can conflict with receive ownership and logging. Adding a UDS library to the environment alone does not integrate it with se.

7.1 Single-frame ReadDataByIdentifier

This template requires the optional endpoint from section 6. It handles a bounded number of ResponsePending replies and rejects other negative responses. The one-second waits are example values, not negotiated ECU timing.

Source: examples/scripts/mcu/07_uds_read_did_sf_template.py

```
"""MCU TEMPLATE: Classic CAN, normal-addressed ISO-TP single frames only."""
FIRMWARE_CONTRACT_CONFIRMED = False
se.check(FIRMWARE_CONTRACT_CONFIRMED, 'Review IDs/DID and ECU timing first')
# Example physical request/response IDs and a fictional two-byte DID.
TX_ID, RX_ID = 0x700, 0x708
DID = 0x1234
# ISO-TP length=3, then UDS ReadDataByIdentifier and its two DID bytes.
se.can.send(TX_ID, [3, 0x22, DID >> 8, DID & 0xFF, 0, 0, 0, 0])
for attempt in range(4):
    frame = se.can.recv(1.0, arbitration_id=RX_ID, extended=False)
    se.check(frame is not None, 'UDS response received')
    se.check(not frame.is_fd and len(frame.data) >= 2, 'Classic CAN data')
    pci = frame.data[0]
    se.check(pci >> 4 == 0, 'Single Frame required; no multi-frame support')
    length = pci & 0x0F
    se.check(1 <= length <= 7 and length < len(frame.data), 'SF length')
    pdu = frame.data[1:1 + length]
    if pdu[0] == 0x7F:
        se.check(len(pdu) == 3 and pdu[1] == 0x22, 'Matching negative reply')
        if pdu[2] == 0x78:
            se.log('Response pending; wait again without resending')
            continue
        raise RuntimeError(f'UDS negative response: NRC 0x{pdu[2]:02X}')
    expected = bytes([0x62, DID >> 8, DID & 0xFF])
    se.check(len(pdu) == 5 and pdu[:3] == expected, 'Two-byte DID response')
    value = int.from_bytes(pdu[3:5], 'big')
    se.log(f'DID 0x{DID:04X} value = 0x{value:04X}')
    break
else:
    raise RuntimeError('Response stayed pending beyond four receive waits')
# This teaching example is not a complete ISO-TP/UDS client or VIN reader.
```

The private automated test peer sends one 7F 22 78 followed by a valid two-byte DID result. This checks the example control flow; it does not qualify an OEM UDS implementation or a physical ECU.

8 Reusable test patterns

Keep a project-specific constants block

```
REQUEST_ID = 0x321
RESPONSE_ID = 0x221
RESPONSE_TIMEOUT_S = 2.0
INTER_REQUEST_MS = 100

def verify_echo(payload):
    se.can.send(REQUEST_ID, payload)
    frame = se.can.recv(RESPONSE_TIMEOUT_S,
                        arbitration_id=RESPONSE_ID, extended=False)
    se.check(frame is not None, "Echo arrived")
    se.check(frame.data == payload, "Echo payload matched")
    return frame

for value in (0x00, 0x55, 0xAA, 0xFF):
    verify_echo(bytes([value] * 8))
    se.sleep(INTER_REQUEST_MS)
```

A practical test-case structure

1. Arrange: connect the correct session, record firmware/database versions and establish fixture state.

2. Act: send one explicit stimulus with a fresh sequence/transaction when supported.

3. Assert: require a matching received response, then validate length, fields, state and timing against named limits.

4. Record: log the measured values and collect an MF4 evidence file separately.

5. Clean up: return outputs/modes to the defined bench state during normal completion; also provide an independent stop/fault recovery plan.

Small helpers instead of hidden parallel traffic

Put reusable conversions and response validation in local Python modules saved next to the test. Keep sequence numbers and failure policy visible. Avoid background threads or a second receive loop unless the runner explicitly manages them; the supported mailbox is owned by one script flow.

9 Logging, replay and timestamps

1. Before Run, press Start Logging in CAN / CAN FD. The logger captures both shared RX and local TX independently of display filters or pause.

2. Run the test. Keep the script revision, firmware version, DBC version, adapter/channel, rate and expected outcome with the result.

3. Stop the script as needed, then Stop Logging and save the MF4. Cancelling the save dialog retains the temporary log for Save Log....

4. Choose File Viewer > Open Logfile. Inspect read-only rows; connect the intended destination to replay RX, TX or both from the selected recorded bus.

5. Replay uses recorded gaps. In a request/response test, replaying both directions can transmit historical ECU responses as tester traffic; choose TX only when that is the intended stimulus.

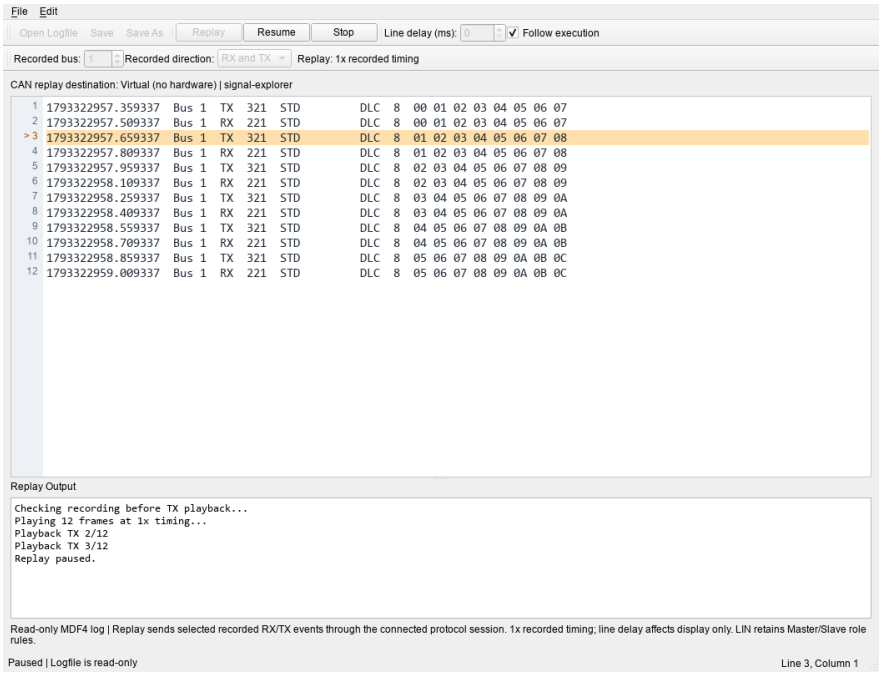


Figure 2: Replay Output and per-row highlighting in log mode.

Line delay differs by mode: Python delay slows real source execution; logfile delay samples highlighted rows without changing transmit timing. Replay Pause/Resume preserves the remaining recorded gaps.

Timestamp correction: Intrepid CAN/LIN/Ethernet now anchor hardware RX intervals to PC time on each connection, matching local TX's epoch. Earlier logs mixing device/PC clocks can have irrecoverably flattened intervals. Create fresh recordings after updating. Preflight rejects selected gaps over 24 hours before any TX; valid gaps are not compressed and there is no Skip gaps option.

9.1 TX lists, channel maps and test isolation

Keep three different artifacts

Artifact	Purpose
Python (.py)	Executable checks, sequencing, response matching and PASS/FAIL logic.
TX list (.json)	Signal Explorer-specific raw/DBC message configuration, channel, period and waveform settings. It is not Python and not a universal interchange format.
DBC / MF4	DBC defines message layouts/scaling. MF4 records RX/TX observations with timestamps and source channels. Save both test configuration and evidence when needed.

Background traffic is independent

Multi-message TX... opens a separate sender. Its running rows can be edited live; accepted edits affect later sends, with an in-flight frame possibly retaining old bytes. The Python editor still locks during a running script so source-line highlighting stays accurate. No Python API was added to create, edit or stop GUI transmit-list rows.

Python Pause/Stop does not pause independent multi-message or main-window traffic. Monitor Pause Exploring and ID filters affect display, not TX or recording. Stop GUI cyclic traffic yourself before a test that requires an otherwise quiet bus. Starting CAN logfile replay stops the main repeat sender and multi-message schedules; disconnecting stops the script and those schedules.

Record and replay the intended source

Start Logging before a script to capture RX/TX from all enabled CAN channels. The MF4 session attachment includes channels with native IDs, labels and recorded BusChannel values. Vector CAN1-CAN4 map to buses 1-4 even for nonadjacent selections; other backends use reported labels or selection order.

For replay, choose one Recorded bus, RX/TX direction and one enabled main TX channel. Every selected replayable observation becomes TX on that destination at best-effort original timing. Replaying both a local TX and its looped-back RX can duplicate the same payload: select the intended direction and source. This is not automatic synchronized multi-channel rebroadcast.

Remember
TX counters report local API acceptance, not peer acknowledgement. Validate the expected RX ID, channel, bytes and transaction fields. Separate hardware clocks are not synchronized by a shared file or by the script runner.

10 Troubleshooting and practical limits

Symptom	Check or next action
NameError: se is not defined	Open/run inside Signal Explorer, not python example.py in a terminal. se is injected by the runner.
No CAN session / Run unavailable	Connect CAN/CAN FD in the main window. The selected LIN/Ethernet tab is not a Python transport. Stop another active script.
Configuration guard failed	Expected for unreviewed MCU templates. Define matching firmware/fixture and update a saved copy before enabling it.
No 221 response	Use Virtual CAN, Inject demo traffic, standard 321 and exactly eight bytes. No physical adapter automatically implements this peer.
Timeout / unexpected reply	Match ID, direction, extended format, length and transaction. Check the peer and bus setup. A local TX is not a response.
IndexError / wrong decoded values	Check frame is not None and length before indexing. Verify endian, signedness, bit position, scale, offset and units.
CAN FD send failed	Connect in FD mode first; verify adapter support, data bitrate and allowed byte count. Classic-only hardware cannot be made FD by a script flag.
Script mailbox overflow	The independent queue holds 2000 events, including unrelated traffic. Long pauses/high input can fill it; reduce load or consume promptly. Failure is explicit.
Pause remains Pausing	A blocking native call has not returned to a checkpoint. Use se.sleep/se.can.recv. Stop rejects further bus API calls and may kill the worker after one second.
Input / local helper import error	input() is unsupported. Save scripts/helpers in the same folder; use the same app environment for optional imports.
Passed after a caught assertion?	A failed se.check still makes the final outcome FAILED. Do not use it for expected exceptions without a deliberate failure policy.
UDS First Frame / other NRC	The supplied example is single-frame only. Match ECU session/timing and implement a proper integrated transport before expanding it.
Timestamp discontinuity	Create a fresh recording after the clock fix. Historical clamped timing cannot be inferred; log opening does not rewrite it.
Outputs remain active after Stop	Stop controls the script, not MCU state. Apply the defined fixture/firmware cleanup procedure.

Native API tests are mocked. The examples do not establish electrical fault tolerance, bus-off recovery, wire-level timing, multi-adapter synchronization or compliance certification.

Appendix: files and further reading

Example source index

File in examples/scripts/mcu/	Use
01_engine_smoke.py	Five EngineData samples; length, decode and range checks.
02_heartbeat_counter.py	Eight counter/period observations with modulo rollover.
03_echo_vectors.py	Zero, FF, alternating and walking-bit eight-byte echoes.
04_can_fd_echo.py	32-byte FD telemetry and eight-byte FD/BRS echo.
05_gpio_adc_template.py	Matching MCU required: GPIO acknowledgement and ADC limits.
06_reset_recovery_template.py	Matching MCU required: boot counter before/after reset.
07_uds_read_did_sf_template.py	Matching endpoint required: limited UDS single-frame DID read.
08_negative_command_template.py	Matching MCU required: documented unsupported-command reply.
09_receive_timeout.py	Virtual-only absence/timeout example with no TX.

Glossary

MCU: microcontroller. **ECU**: electronic control unit. **DLC**: data length code; the API presents a byte count. **DBC**: CAN message/signal database. **DID**: UDS DataIdentifier. **NRC**: negative response code. **ISO-TP**: diagnostic transport on CAN. **SF**: Single Frame. **Transaction**: application field used to correlate a response with a request.

Primary technical references

[R1] py-uds: diagnostic services and ReadDataByIdentifier

[R2] Linux kernel: ISO 15765-2 (ISO-TP), framing and addressing

[R3] py-uds: diagnostic messages and negative responses

[R4] Python language tutorial

UDS/ISO-TP references were checked October 29, 2026. They explain protocols, not an implemented library integration. Application API behavior is defined by src/signal_explorer/scripting/, docs/scripting.md and docs/file-viewer.md.

Appendix: validation and next steps

What was checked for this edition

196 automated tests cover the application and guide examples, including channel-aware routing and live TX lists. All nine tutorial sources compile. The five virtual examples run through the actual separate-process scripting runner with the built-in demo. The four MCU templates are checked both for their default no-transmit guard and against a private virtual mock firmware peer after enabling the contract flag in the test harness. The UDS mock emits ResponsePending then a final positive reply; an out-of-range ADC value is also checked to fail the GPIO/ADC test.

These mock peers live in tests/test_scripting_examples.py and are not a second firmware simulator exposed by the GUI. The test harness does not change the shipped templates' default guard. No new physical adapter/MCU qualification is claimed by these software checks.

Run checks from the project root

```
.\.venv\Scripts\python.exe -m pytest tests -q
# Only the guide examples (PowerShell line continuation):
.\.venv\Scripts\python.exe -m pytest `
    tests/test_scripting_examples.py -q
```

The backtick continues the second PowerShell command onto the next line. See the Install Guide for environment/Intrepid optional package setup. Documentation regeneration uses reportlab and pypdf; those are not additional runtime dependencies for the examples.

A useful first MCU test suite

1. Start with one status/heartbeat arrival check and a known firmware-version response.

2. Add correlation, length and signal range checks for one benign request/response.

3. Add output/ADC checks with independent fixture measurement and defined cleanup.

4. Add reboot recovery, expected command rejection and explicit timeouts.

5. Store test and firmware revisions alongside MF4 evidence, then verify on the intended physical adapter.

Remember
SENT, LIN/Ethernet Python scripting, automatic CAPL translation, complete UDS/ISO-TP, fixture power control and integrated test-report export remain future work. Standard Python can write your own report files, but the application does not supply a report API in this release.

Revision 1.1 - November 2, 2026. Adds enabled-channel tests, TX-list interaction and multi-channel evidence. Companion to User and Install Guide revision 1.5.